

scripts/load_podman_secrets.sh — operator guide

Revision: 2 **Last modified:** 2026-07-01T12:00:00Z **Status:** DESIGN / config-plan companion (§11.4.18). Live secret-injection + leak-free proof is owed to P10 (§11.4.6).

Companion to config/security/README.md §1–§2. This is the external user guide §11.4.18 requires alongside the in-source documentation block at the top of the script. (Project-wide note: the other scripts/*.sh predate this convention and their companions are tracked docs-sync backfill debt — P11.)

Overview

Idempotently creates the **named**, **rootless** Podman secrets the dynamic-routing stack references (Postgres password, control-API mTLS cert+key+**client-CA**, WireGuard tunnel key, Squid httpasswd) from operator-provisioned, **gitignored** source files. The script carries **no secret values** (§11.4.10) — only the mapping of secret-NAME → expected gitignored SOURCE-PATH. Containers then reference the secrets by name; the value is mounted into the container tmpfs at runtime and never reaches an image layer, a compose file, or git.

Prerequisites

- **Rootless Podman** on PATH (§11.4.161). The script **refuses to run as root** (exit 2).
- Operator-provisioned source files (out-of-band, real values, never in git):
 - secrets/pg_password, secrets/api_cert.pem, secrets/api_key.pem, secrets/api_ca.pem (the control-API mTLS client CA — the trust anchor for the ClientCAs verify pool), secrets/vpn_wg_key (under a chmod 700 secrets/, each chmod 600)
 - config/httpasswd (bcrypt hashes via httpasswd -B, never plaintext)
 - All of the above are excluded by .gitignore.

Usage examples

```
# Preview what would be created, change nothing:  
scripts/load_podman_secrets.sh --dry-run
```

```
# Create any absent secrets (safe to re-run – existing secrets are  
    kept):  
scripts/load_podman_secrets.sh
```

```
# Force-recreate (rotate) secrets that already exist:  
scripts/load_podman_secrets.sh --replace
```

Names default to the *_SECRET values in .env.example; override any via the matching environment variable (e.g. PG_PASSWORD_SECRET, SECRETS_DIR, HTPASSWD_FILE).

Edge cases

- **Missing source file** → that secret is **SKIP**ped (not created); the script exits 3 with a per-secret summary so the operator knows exactly what to provision. Never creates an empty/placeholder secret.
- **Runtime absent / running as root** → exit 2, nothing created.
- **Loose source perms** (not 600/400/640/440) → a **WARN** (not a failure); the contents are never printed (§11.4.10).
- **Re-run after a partial provision** → idempotent: already-present secrets are reported EXISTS and kept unless --replace is given.

Internal behaviour

- set -eu; counters use VAR=\$((VAR+1)) (always ≥ 1) so the increment never trips the set -e arithmetic-zero abort (the existing-test B4 antipattern).
- Touches **only** helixproxy_*-named secrets — never the operator's own containers, the wg0-mullvad / lava-* interfaces, or unrelated secrets (§11.4.174).
- secret_exists uses secret exists with an inspect fallback for runtimes lacking the subcommand.

Control-API secret contract (name → mounted path)

The control-API (control-plane/cmd/api) is served over **mTLS** and loads its cert, key, and **client-CA** from FILE PATHS — never embedded key material (§11.4.10). Three layers MUST agree, and this script provisions the middle one:

Layer	Variable(s)	Value
Secret name (.env.example, this script)	CONTROL_API_TLS_CERT_SECRET / CONTROL_API_TLS_KEY_SECRET / CONTROL_API_CLIENT_CA_SECRET	helixproxy_api_cert / helixproxy_api_key / helixproxy_api_client_ca
Mounted path (.env.example, read by the code)	CONTROL_API_TLS_CERT / CONTROL_API_TLS_KEY / CONTROL_API_TLS_CLIENT_CA	/run/secrets/ helixproxy_api_cert / ... _api_key / ... _api_client_ca
Listen address	CONTROL_API_ADDR	:58080 (code default)

The name → path bridge: this script creates a Podman secret named <name>; the compose api service mounts it read-only at /run/secrets/<name>; the control-API reads that path. So CONTROL_API_CLIENT_CA_SECRET=helixproxy_api_client_ca (source secrets/api_ca.pem) is mounted at /run/secrets/helixproxy_api_client_ca, which is exactly the CONTROL_API_TLS_CLIENT_CA path the code reads. The client-CA is the trust anchor for the ClientCAs pool: the control-API is **fail-closed** (RequireAndVerifyClientCert) and refuses to start if any of the three paths is empty or unreadable.

Scope: wiring the compose **api service** itself (build the cmd/api binary, mount these three secrets at the paths above, expose the port) is **P10 deployment work** — this script + .env.example define the coherent name/path **contract**; they do not boot the service.

Related

- config/security/README.md — the secret-reference + zero-trust model.
- config/squid/templates/auth.conf.tmpl — consumes helixproxy_proxy_htpasswd.
- .env.example — declares the *_SECRET **names**, the CONTROL_API_TLS_* mounted **paths**, and CONTROL_API_ADDR (never any secret values).

- Spec §12 (Security & secrets): docs/superpowers/specs/2026-06-30-vpn-aware-proxy-extension-design.md.

Last verified

2026-07-01 — sh -n + bash -n parse-clean; pre-store secret-value scan CLEAN. Live secret creation against a real rootless Podman is exercised in **P10**.